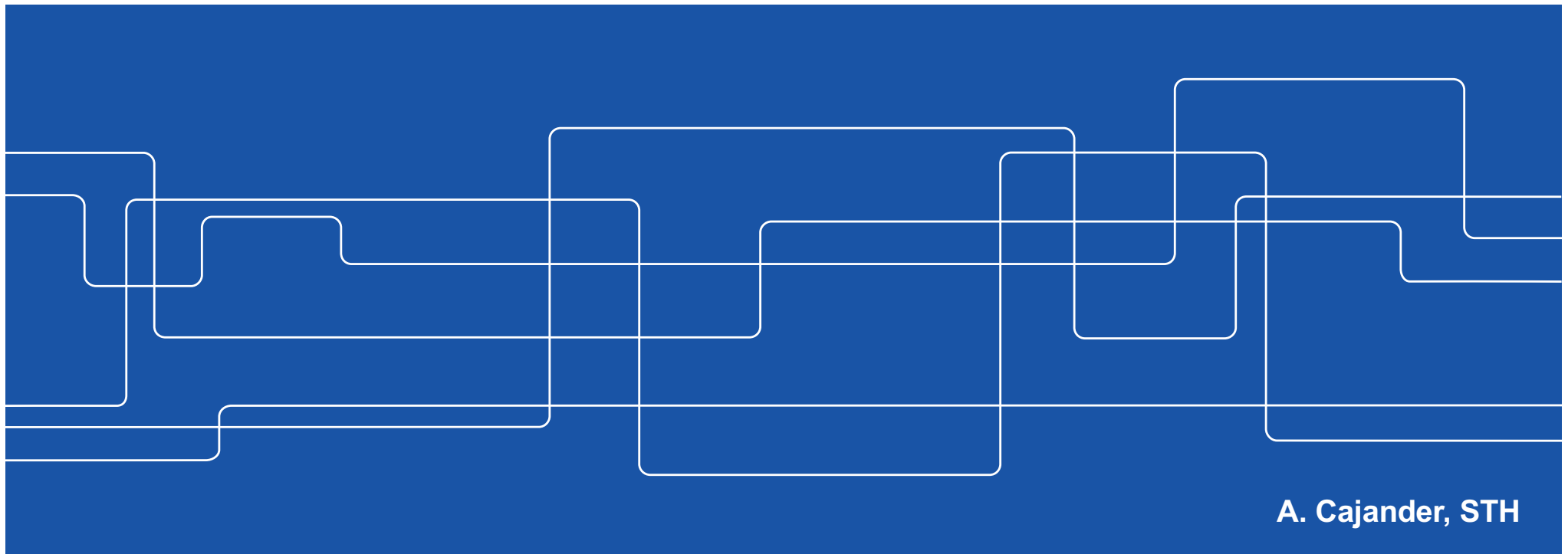




HE1025 Operativsystem

F8: Kommunikation – Communication & distributed systems



A. Cajander, STH



Kursen...

F1: V#1 Processorn.	Kap (1) 2..4 & 6
Ö1: V#2 c Pekare...	Kap 14 & c-boken
F2: V#3 Schemaläggning	Kap 7..9,(10-11), RTOS
L1: V#4 FreeRTOS	Lab #1 Handledning
F3: V#5 Segmentation	Kap 12..13, 15..17
Ö2: V#6 Buffer	t.b.d
F4: V#7 Paging	Kap 18-22, (23-24)
L2: V#8 Paging	Lab #2 Handledning
F5: C#1 Locks & CV	Kap (25) 26..27 & 28..29.1
Ö3: C#2 LINUX	Kap 5 (39)
F6: C#3 Semaphores	Kap 30..33, (34)
L3: C#4 Barberare	Lab #3 Handledning
F7: P#1 Storage	Kap (35,) 36, 40, 42 (37..39, 41 & 43..46)
Ö4: P#2 SD Card/DMA	Orientering
F8: P#3 Communication	Kap (47, 39) & 48, (49..51)
L4: P#4 Client/Server	Lab #4 Handledning

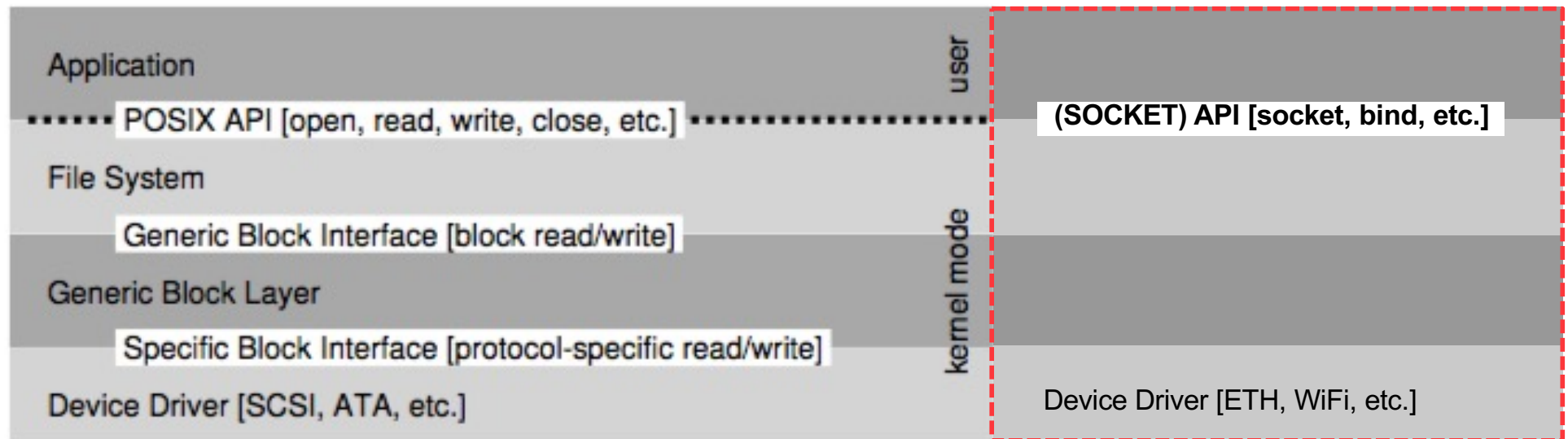
TEN1: 4+4p/4:e-del, minst 4p per 4:e-del för E!

LAB1: Närvaro L1..4



Kommunikation...

En socket är en abstraktion av en mer eller mindre bra förbindelse mellan två ändpunkter som kan vara i samma/eller olika maskiner!



Förbindelsen kan uppenbart användas för **IP/TC**...

...men också i en förlängning till att bygga ett distribuerat operativsys...

...där kanske **tangentbord/skärm** finns på en annan plats...

...eller **filsystemet** finns på en/flera andra platser...

...eller efterfrågade **tjänster** finns på en/flera andra platser!

Det viktigaste är att komma ihåg att information kan gå förlorad...

...men först sammanfattar vi vad vi vet om Inter Proc/Thread Com!



Inter Process/Thread Communication

- **Shared memory** permits processes to communicate by simply reading and writing to a specified memory location.
- **Mapped memory** is similar to shared memory, except that it is associated with a file in the filesystem.
- **Pipes** permit sequential communication from one process to a related process.
- **FIFOs** are similar to pipes, except that unrelated processes can communicate because the FIFO is given a name in the filesystem.
- **Sockets** support communication between unrelated processes, even on different computers.



Shared memory

Snabbaste mekanismen för IPC...

...undviker kärnanrop och att datat behöver kopieras...

...access måste dock koordineras via t.ex. locks!

Linux: `shmget(key, size, flg)` [SHared Memory GET]

key används av "andra" processer för att erhålla
access till allokerad minnesarea. (Flg = flaggor.)

```
int segment_id = shmget (shm_key, getpagesize (),  
                        IPC_CREAT | S_IRUSR | S_IWUSER);
```

Därefter måste segmentet kopplas via `shmat(tach)` så att det
blir tillgängligt någonstans i processens adressrymd!

(Minnet måste återlämnas via `shmd(e)t(tach)`!)



Shared memory

```
#include <stdio.h>
#include <sys/shm.h>
#include <sys/stat.h>
```

```
int main ()
{
    int segment_id;
    char* shared_memory;
    struct shmid_ds shmbuffer;
    int segment_size;
    const int shared_segment_size = 0x6400;

    /* Allocate a shared memory segment. */
    segment_id = shmget (IPC_PRIVATE, shared_segment_size,
                        IPC_CREAT | IPC_EXCL | S_IRUSR | S_IWUSR);

    /* Attach the shared memory segment. */
    shared_memory = (char*) shmat (segment_id, 0, 0);
    printf ("shared memory attached at address %p\n", shared_memory);
    /* Determine the segment's size. */
    shmctl (segment_id, IPC_STAT, &shmbuffer);
    segment_size = shmbuffer.shm_segsz;
    printf ("segment size: %d\n", segment_size);
    /* Write a string to the shared memory segment. */
    sprintf (shared_memory, "Hello, world.");
    /* Detach the shared memory segment. */
    shmdt (shared_memory);

    /* Reattach the shared memory segment, at a different address. */
    shared_memory = (char*) shmat (segment_id, (void*) 0x50000000, 0);
    printf ("shared memory reattached at address %p\n", shared_memory);
    /* Print out the string from shared memory. */
    printf ("%s\n", shared_memory);
    /* Detach the shared memory segment. */
    shmdt (shared_memory);

    /* Deallocate the shared memory segment. */
    shmctl (segment_id, IPC_RMID, 0);

    return 0;
}
```

```
% ipcs -m
```

```
----- Shared Memory Segments -----
key          shmid    owner    perms    bytes    nattch   status
0x00000000  1627649    user     640      25600     0
```

```
% ipcrm shm 1627649
```

0x0000

Hello World.

25600B



Shared memory

```
#include <stdio.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main ()
{
    int segment_id;
    char* shared_memory;
    struct shm_id_ds shmbuffer;
    int segment_size;
    const int shared_segment_size = 0x6400;

    /* Allocate a shared memory segment. */
    segment_id = shmget (IPC_PRIVATE, shared_segment_size,
                        IPC_CREAT | IPC_EXCL | S_IRUSR | S_IWUSR);

    /* Attach the shared memory segment. */
    shared_memory = (char*) shmat (segment_id, 0, 0);
    printf ("shared memory attached at address %p\n", shared_memory);
    /* Determine the segment's size. */
    shmctl (segment_id, IPC_STAT, &shmbuffer);
    segment_size = shmbuffer.shm_segsz;
    printf ("segment size: %d\n", segment_size);
    /* Write a string to the shared memory segment. */
    sprintf (shared_memory, "Hello, world.");
    /* Detach the shared memory segment. */
    shmdt (shared_memory);

    /* Reattach the shared memory segment, at a different address. */
    shared_memory = (char*) shmat (segment_id, (void*) 0x5000000, 0);
    printf ("shared memory reattached at address %p\n", shared_memory);
    /* Print out the string from shared memory. */
    printf ("%s\n", shared_memory);
    /* Detach the shared memory segment. */
    shmdt (shared_memory);

    /* Deallocate the shared memory segment. */
    shmctl (segment_id, IPC_RMID, 0);

    return 0;
}
```

```
% ipcs -m
```

```
----- Shared Memory Segments -----
key          shmid    owner    perms    bytes    nattch   status
0x00000000  1627649    user     640      25600    0
```

```
% ipcrm shm 1627649
```

0x5000000

Hello World.

25600B



Mapped Memory

Läser in en namngiven fil till processens adressrymd...

...läs/skrivning går snabbt...

...os:et hanterar kopplingen till den underliggande filen!

`mmap(...)` = läser in (del av) en fil till önskad plats i minnet!

`msync(...)` = säkerställer att alla tar del av en write.

`munmap(...)` = släpper kopplingen till filen.



Mapped memory

```
#include <stdlib.h>
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>
#define FILE_LENGTH 0x100

/* Return a uniformly random number in the range [low,high]. */
int random_range (unsigned const low, unsigned const high)
{
    unsigned const range = high - low + 1;
    return low + (int) (((double) range) * rand () / (RAND_MAX + 1.0))
}

int main (int argc, char* const argv[])
{
    int fd;
    void* file_memory;

    /* Seed the random number generator. */
    srand (time (NULL));

    /* Prepare a file large enough to hold an unsigned integer. */
    fd = open (argv[1], O_RDWR | O_CREAT, S_IRUSR | S_IWUSR);
    lseek (fd, FILE_LENGTH+1, SEEK_SET);
    write (fd, "", 1);
    lseek (fd, 0, SEEK_SET);

    /* Create the memory mapping. */
    file_memory = mmap (0, FILE_LENGTH, PROT_WRITE, MAP_SHARED, fd, 0);
    close (fd);

    /* Write a random integer to memory-mapped area. */
    sprintf((char*) file_memory, "%d\n", random_range (-100, 100));
    /* Release the memory (unnecessary because the program exits). */
    munmap (file_memory, FILE_LENGTH);

    return 0;
}
```



Mapped memory

```
#include <stdlib.h>
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>
#define FILE_LENGTH 0x100
```

```
/* Return a uniformly random number in the range [low,high].
```

```
int random_range (unsigned const low, unsigned const high)
{
    unsigned const range = high - low + 1;
    return low + (int) (((double) range) * rand () / (RAND_MAX +
```

```
int main (int argc, char* const argv[])
{
```

```
    int fd;
    void* file_memory;
```

```
    /* Seed the random number generator. */
    srand (time (NULL));
```

```
    /* Prepare a file large enough to hold an unsigned integer. */
    fd = open (argv[1], O_RDWR | O_CREAT, S_IRUSR | S_IWUSR);
    lseek (fd, FILE_LENGTH+1, SEEK_SET);
    write (fd, "", 1);
    lseek (fd, 0, SEEK_SET);
```

```
    /* Create the memory mapping. */
    file_memory = mmap (0, FILE_LENGTH, PROT_WRITE, MAP_SHARED, fd, 0);
    close (fd);
    /* Write a random integer to memory-mapped area. */
    sprintf((char*) file_memory, "%d\n", random_range (-100, 100));
    /* Release the memory (unnecessary because the program exits). */
    munmap (file_memory, FILE_LENGTH);
```

```
    return 0;
}
```

```
#include <stdlib.h>
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>
#define FILE_LENGTH 0x100
```

```
int main (int argc, char* const argv[])
{
```

```
    int fd;
    void* file_memory;
    int integer;
```

```
    /* Open the file. */
    fd = open (argv[1], O_RDWR, S_IRUSR | S_IWUSR);
    /* Create the memory mapping. */
    file_memory = mmap (0, FILE_LENGTH, PROT_READ | PROT_WRITE,
                        MAP_SHARED, fd, 0);
```

```
    close (fd);
```

```
    /* Read the integer, print it out, and double it. */
    scanf (file_memory, "%d", &integer);
    printf ("value: %d\n", integer);
    sprintf ((char*) file_memory, "%d\n", 2 * integer);
    /* Release the memory (unnecessary because the program exits). */
    munmap (file_memory, FILE_LENGTH);
```

```
    return 0;
```

```
}
```



Mapped memory

```
#include <stdlib.h>
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>
#define FILE_LENGTH 0x100

/* Return a uniformly random number in the range [low,high].

int random_range (unsigned const low, unsigned const high)
{
    unsigned const range = high - low + 1;
    return low + (int) (((double) range) * rand () / (RAND_MAX + 1));
}

int main (int argc, char* const argv[])
{
    int fd;
    void* file_memory;

    /* Seed the random number generator. */
    srand (time (NULL));

    /* Prepare a file large enough to hold an unsigned integer. */
    fd = open (argv[1], O_RDWR | O_CREAT, S_IRUSR | S_IWUSR);
    lseek (fd, FILE_LENGTH+1, SEEK_SET);
    write (fd, "", 1);
    lseek (fd, 0, SEEK_SET);

    /* Create the memory mapping. */
    file_memory = mmap (0, FILE_LENGTH, PROT_WRITE, MAP_SHARED, fd, 0);
    close (fd);

    /* Write a random integer to memory-mapped area. */
    sprintf((char*) file_memory, "%d\n", random_range (-100, 100));
    /* Release the memory (unnecessary because the program exits). */
    munmap (file_memory, FILE_LENGTH);

    return 0;
}
```

```
#include <stdlib.h>
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>
#define FILE_LENGTH 0x100

int main (int argc, char* const argv[])
{
    int fd;
    void* file_memory;
    int integer;

    /* Open the file. */
    fd = open (argv[1], O_RDWR, S_IRUSR | S_IWUSR);
    /* Create the memory mapping. */
    file_memory = mmap (0, FILE_LENGTH, PROT_READ | PROT_WRITE,
                        MAP_SHARED, fd, 0);

    close (fd);

    /* Read the integer, print it out, and double it. */
    scanf (file_memory, "%d", &integer);
    printf ("value: %d\n", integer);
    sprintf ((char*) file_memory, "%d\n", 2 * integer);
    /* Release the memory (unnecessary because the program exits). */
    munmap (file_memory, FILE_LENGTH);

    return 0;
}
```

```
% ./mmap-write /tmp/integer-file
% cat /tmp/integer-file
42
% ./mmap-read /tmp/integer-file
value: 42
% cat /tmp/integer-file
84
```



Pipes

Pipes är en (begränsad) seriell buffert för enkelriktad kom!

...bufferten garanterar att ordningen bibehålls...

...blockerar skriv/läs om full/tom...

...fungerar enbart mellan trådar eller relaterade proc!

`pipe(int fds[])` = skapar pipe och returnera r & w fds...

Vanligt att konv. fds till streams via `fdopen(fds[0/1], "r/w");`

Om den används tillsammans med fork...

...glöm inte bort att stänga ej använda ändar!

(Varför är det viktigt?)



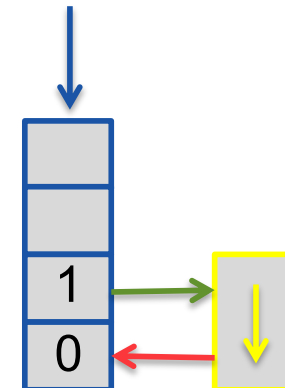
```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
```

```
int main ()
{
    int fds[2];
    pid_t pid;

    /* Create a pipe. File descriptors for the two ends of the pipe are
       placed in fds. */
    pipe (fds);
    /* Fork a child process. */
    pid = fork ();
    if (pid == (pid_t) 0) {
        FILE* stream;
        /* This is the child process. Close our copy of the write end of
           the file descriptor. */
        close (fds[1]);
        /* Convert the read file descriptor to a FILE object, and read
           from it. */
        stream = fdopen (fds[0], "r");
        reader (stream);
        close (fds[0]);
    }
    else {
        /* This is the parent process. */
        FILE* stream;
        /* Close our copy of the read end of the file descriptor. */
        close (fds[0]);
        /* Convert the write file descriptor to a FILE object, and write
           to it. */
        stream = fdopen (fds[1], "w");
        writer ("Hello, world.", 5, stream);
        close (fds[1]);
    }

    return 0;
}
```

Parent fds table





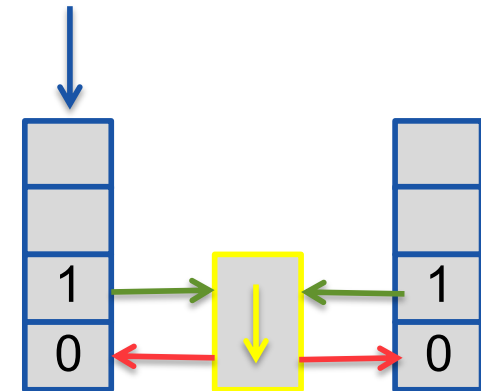
```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
```

```
int main ()
{
    int fds[2];
    pid_t pid;

    /* Create a pipe. File descriptors for the two ends of the pipe are
       placed in fds. */
    pipe (fds);
    /* Fork a child process. */
    pid = fork ();
    if (pid == (pid_t) 0) {
        FILE* stream;
        /* This is the child process. Close our copy of the write end of
           the file descriptor. */
        close (fds[1]);
        /* Convert the read file descriptor to a FILE object, and read
           from it. */
        stream = fdopen (fds[0], "r");
        reader (stream);
        close (fds[0]);
    }
    else {
        /* This is the parent process. */
        FILE* stream;
        /* Close our copy of the read end of the file descriptor. */
        close (fds[0]);
        /* Convert the write file descriptor to a FILE object, and write
           to it. */
        stream = fdopen (fds[1], "w");
        writer ("Hello, world.", 5, stream);
        close (fds[1]);
    }

    return 0;
}
```

Parent fds table



Child fds table



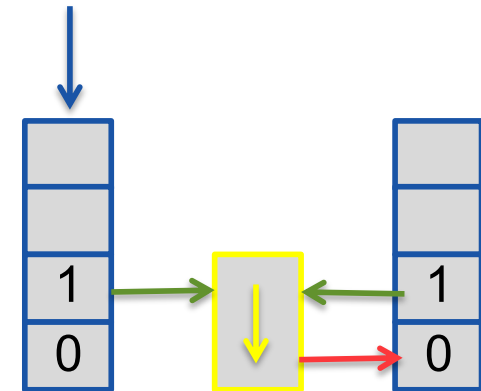
```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>

int main ()
{
    int fds[2];
    pid_t pid;

    /* Create a pipe. File descriptors for the two ends of the pipe are
       placed in fds. */
    pipe (fds);
    /* Fork a child process. */
    pid = fork ();
    if (pid == (pid_t) 0) {
        FILE* stream;
        /* This is the child process. Close our copy of the write end of
           the file descriptor. */
        close (fds[1]);
        /* Convert the read file descriptor to a FILE object, and read
           from it. */
        stream = fdopen (fds[0], "r");
        reader (stream);
        close (fds[0]);
    }
    else {
        /* This is the parent process. */
        FILE* stream;
        /* Close our copy of the read end of the file descriptor. */
        close (fds[0]);
        /* Convert the write file descriptor to a FILE object, and write
           to it. */
        stream = fdopen (fds[1], "w");
        writer ("Hello, world.", 5, stream);
        close (fds[1]);
    }

    return 0;
}
```

Parent fds table



Child fds table



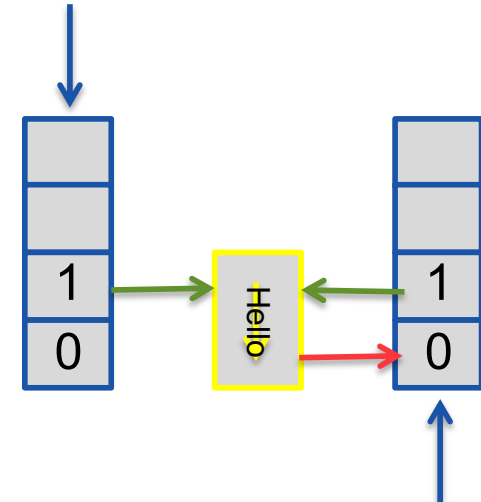
```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
```

```
int main ()
{
    int fds[2];
    pid_t pid;

    /* Create a pipe. File descriptors for the two ends of the pipe are
       placed in fds. */
    pipe (fds);
    /* Fork a child process. */
    pid = fork ();
    if (pid == (pid_t) 0) {
        FILE* stream;
        /* This is the child process. Close our copy of the write end of
           the file descriptor. */
        close (fds[1]);
        /* Convert the read file descriptor to a FILE object, and read
           from it. */
        stream = fdopen (fds[0], "r");
        reader (stream);
        close (fds[0]);
    }
    else {
        /* This is the parent process. */
        FILE* stream;
        /* Close our copy of the read end of
           close (fds[0]);
        /* Convert the write file descriptor
           to it. */
        stream = fdopen (fds[1], "w");
        writer ("Hello, world.", 5, stream);
        close (fds[1]);
    }

    return 0;
}
```

Parent fds table



Child fds table

```
/* Write COUNT copies of MESSAGE to STREAM, pausing for a second
   between each. */

void writer (const char* message, int count, FILE* stream)
{
    for (; count > 0; --count) {
        /* Write the message to the stream, and send it off immediately
           fprintf (stream, "%s\n", message);
           fflush (stream);
        /* Snooze a while. */
        sleep (1);
    }
}
```



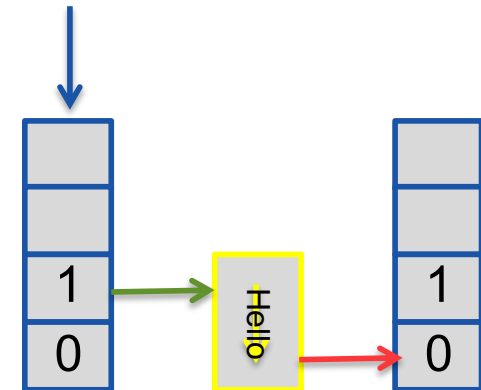
```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>

int main ()
{
    int fds[2];
    pid_t pid;

    /* Create a pipe. File descriptors for the two ends of the pipe are
       placed in fds. */
    pipe (fds);
    /* Fork a child process. */
    pid = fork ();
    if (pid == (pid_t) 0) {
        FILE* stream;
        /* This is the child process. Close our copy of the write end of
           the file descriptor. */
        close (fds[1]);
        /* Convert the read file descriptor to a FILE object, and read
           from it. */
        stream = fdopen (fds[0], "r");
        reader (stream);
        close (fds[0]);
    }
    else {
        /* This is the parent process. */
        FILE* stream;
        /* Close our copy of the read end of the file descriptor. */
        close (fds[0]);
        /* Convert the write file descriptor to a FILE object, and write
           to it. */
        stream = fdopen (fds[1], "w");
        writer ("Hello, world.", 5, stream);
        close (fds[1]);
    }

    return 0;
}
```

Parent fds table



Child fds table



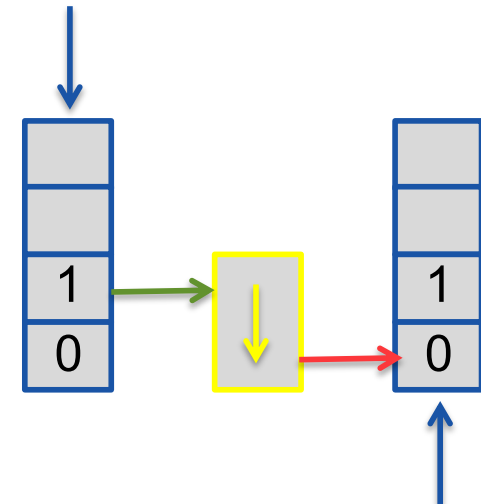
```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
```

```
int main ()
{
    int fds[2];
    pid_t pid;

    /* Create a pipe. File descriptors for the two ends of the pipe are
       placed in fds. */
    pipe (fds);
    /* Fork a child process. */
    pid = fork ();
    if (pid == (pid_t) 0) {
        FILE* stream;
        /* This is the child process. Close our copy of the write end of
           the file descriptor. */
        close (fds[1]);
        /* Convert the read file descriptor to a FILE object, and read
           from it. */
        stream = fdopen (fds[0], "r");
        reader (stream);
        close (fds[0]);
    }
    else {
        /* This is the parent process. */
        FILE* stream;
        /* Close our copy of the read end of
           close (fds[0]);
        /* Convert the write file descriptor
           to it. */
        stream = fdopen (fds[1], "w");
        writer ("Hello, world.", 5, stream);
        close (fds[1]);
    }

    return 0;
}
```

Parent fds table



Child fds table

5 * Hello, world.

```
/* Read random strings from the stream as long as possible. */
void reader (FILE* stream)
{
    char buffer[1024];
    /* Read until we hit the end of the stream. fgets reads until
       either a newline or the end-of-file. */
    while (!feof (stream)
           && !ferror (stream)
           && fgets (buffer, sizeof (buffer), stream) != NULL)
        fputs (buffer, stdout);
}
```



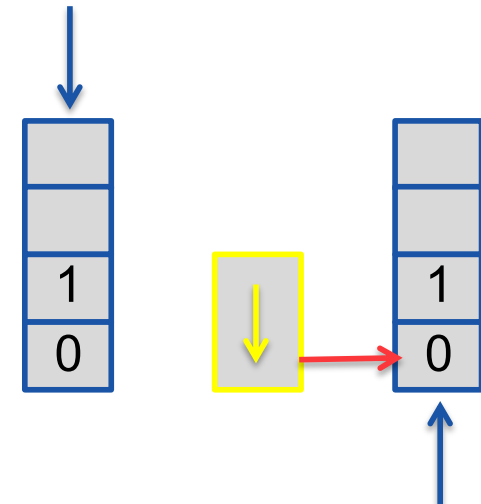
```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
```

```
int main ()
{
    int fds[2];
    pid_t pid;

    /* Create a pipe. File descriptors for the two ends of the pipe are
       placed in fds. */
    pipe (fds);
    /* Fork a child process. */
    pid = fork ();
    if (pid == (pid_t) 0) {
        FILE* stream;
        /* This is the child process. Close our copy of the write end of
           the file descriptor. */
        close (fds[1]);
        /* Convert the read file descriptor to a FILE object, and read
           from it. */
        stream = fdopen (fds[0], "r");
        reader (stream);
        close (fds[0]);
    }
    else {
        /* This is the parent process. */
        FILE* stream;
        /* Close our copy of the read end of
           close (fds[0]);
        /* Convert the write file descriptor
           to it. */
        stream = fdopen (fds[1], "w");
        writer ("Hello, world.", 5, stream);
        close (fds[1]);
    }

    return 0;
}
```

Parent fds table



Child fds table

Hello, world.

```
/* Read random strings from the stream as long as possible. */
void reader (FILE* stream)
{
    char buffer[1024];
    /* Read until we hit the end of the stream. fgets reads until
       either a newline or the end-of-file. */
    while (!feof (stream)
           && !ferror (stream)
           && fgets (buffer, sizeof (buffer), stream) != NULL)
        fputs (buffer, stdout);
}
```

EOF!



```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>

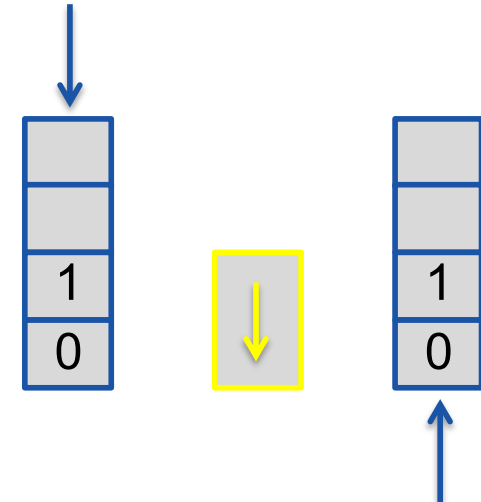
int main ()
{
    int fds[2];
    pid_t pid;

    /* Create a pipe. File descriptors for the two ends of the pipe are
       placed in fds. */
    pipe (fds);
    /* Fork a child process. */
    pid = fork ();
    if (pid == (pid_t) 0) {
        FILE* stream;
        /* This is the child process. Close our copy of the write end of
           the file descriptor. */
        close (fds[1]);
        /* Convert the read file descriptor to a FILE object, and read
           from it. */
        stream = fdopen (fds[0], "r");
        reader (stream);
        close (fds[0]);
    }
    else {
        /* This is the parent process. */
        FILE* stream;
        /* Close our copy of the read end of the file descriptor. */
        close (fds[0]);
        /* Convert the write file descriptor to a FILE object, and write
           to it. */
        stream = fdopen (fds[1], "w");
        writer ("Hello, world.", 5, stream);
        close (fds[1]);
    }

    return 0;
}
```



Parent fds table



Child fds table

Hello, world.



Pipes

Trick: Använd `dup2(fd, STDIN_FILENO)` för att stänga standard in och öppna den igen som en kopia av `fd`. T.ex: `scanf(..)` kommer nu att läsa från `fd` i stället för från tangentbordsbufferten!

Speciellt användbart tillsammans med `execlp(...)` som "byter ut" en forkad process mot något annat program som då tror sig läsa från standar in!

```
pid = fork ();
if (pid == (pid_t) 0) {
    /* This is the child process.  Close our copy of the write end of
       the file descriptor.  */
    close (fds[1]);
    /* Connect the read end of the pipe to standard input.  */
    dup2 (fds[0], STDIN_FILENO);
    /* Replace the child process with the "sort" program.  */
    execlp ("sort", "sort", 0);
}
```



FIFOs (aka Named PIPEs)

mkfifo(...) = skapar en namngiven FIFO(/PIPE)...

...efter det användes file-relaterad I/O funktioner som vanligt!

```
int fd = open (fifo_path, O_WRONLY);  
write (fd, data, data_length);  
close (fd);
```

```
FILE* fifo = fopen (fifo_path, "r");  
fscanf (fifo, "%s", buffer);  
fclose (fifo);
```

Det får finnas flera läsare & skrivare samtidigt!

Sockets (dubbelriktad kommunikation, orelaterade proc)

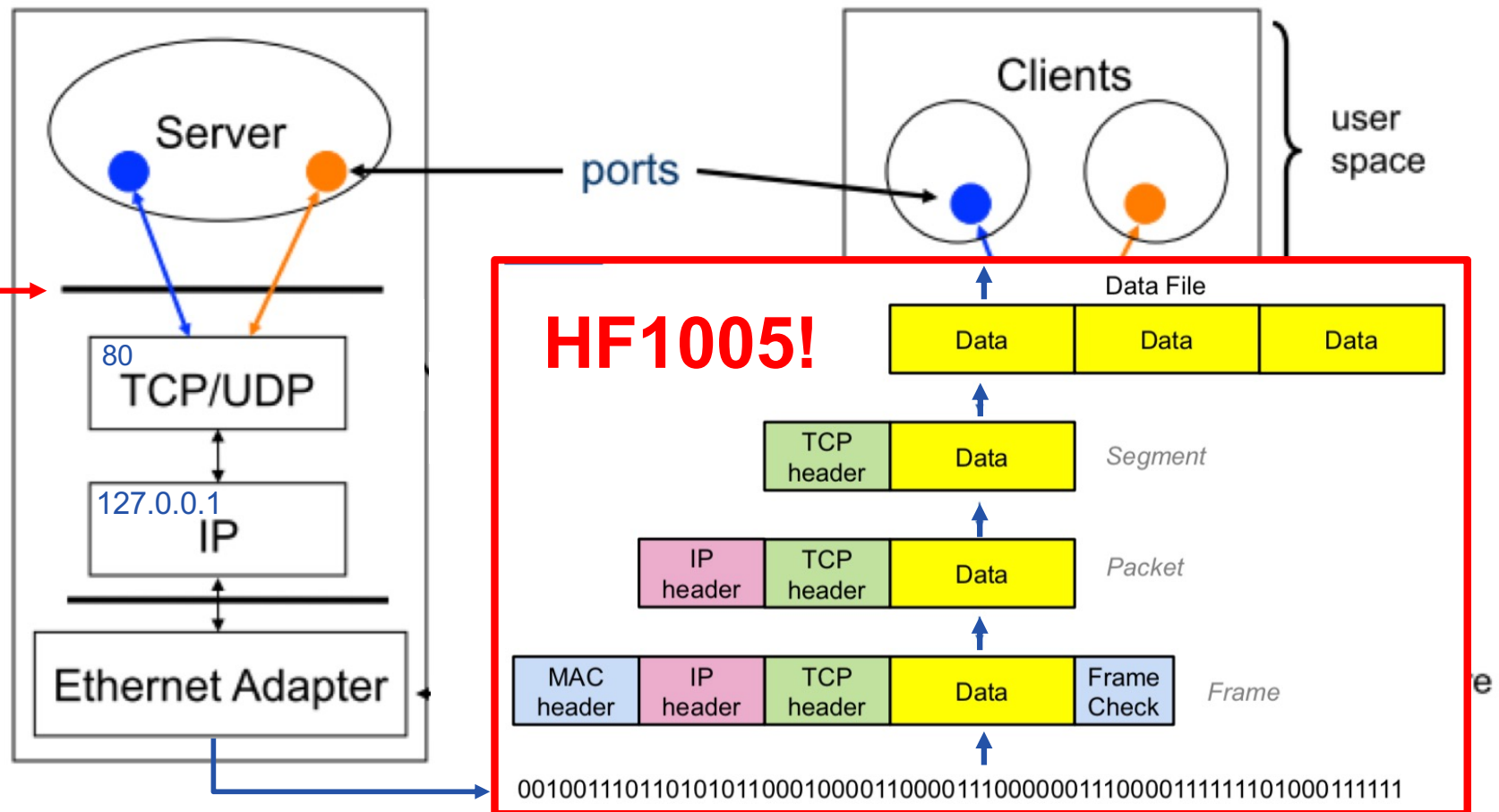
LAB4

Socket
API!

OS

1000-tals
PersonÅr

Device
driver

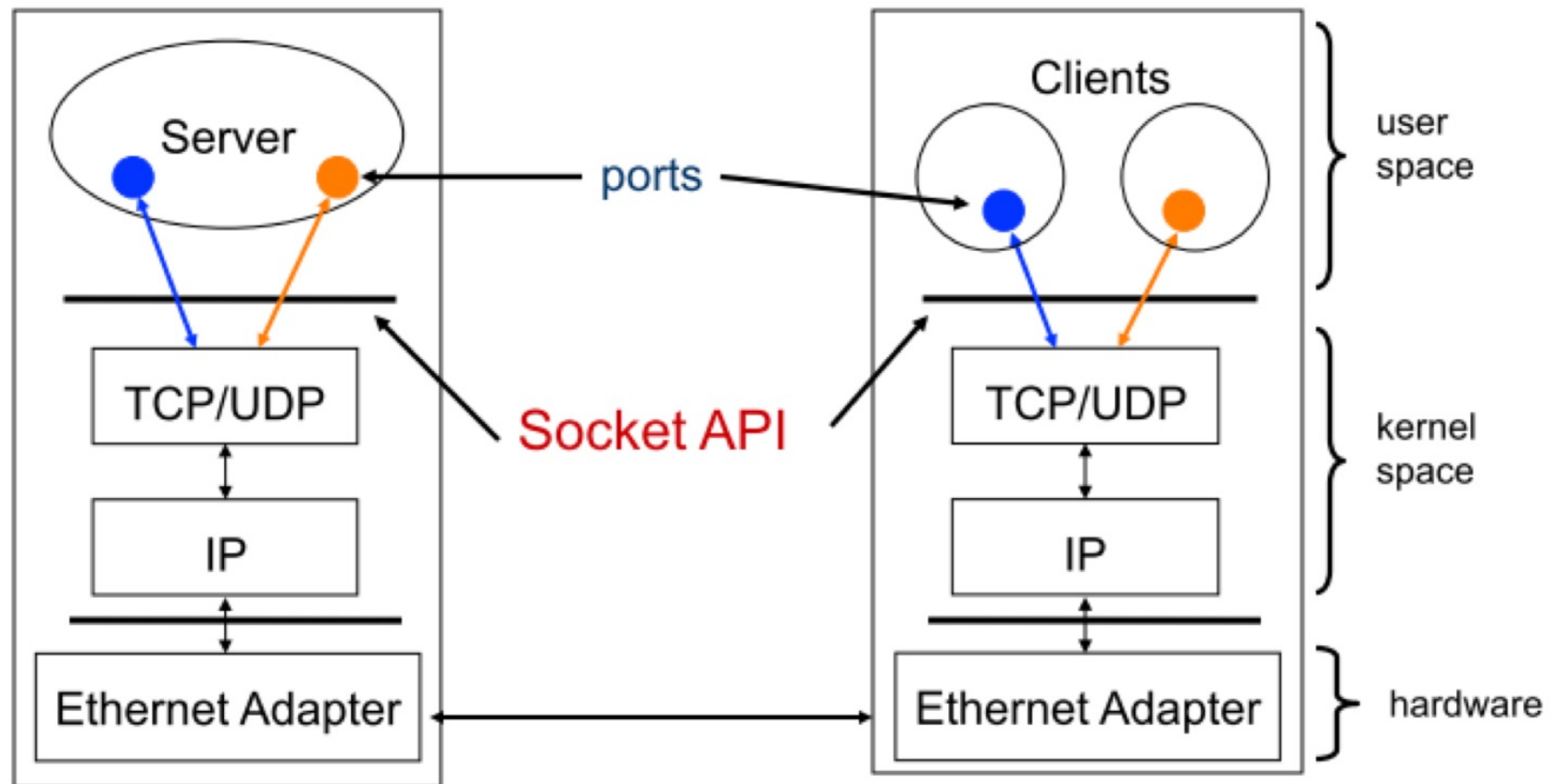


En enskild dator kan identifieras via dess globalt unika ip-adress!
 En enskild tjänst på den datorn via ett så kallat port-nummer!
 (En del vanliga tjänster har så kallat allmänt kända portnummer)

Sockets

Kommer du ihåg?!

LAB4

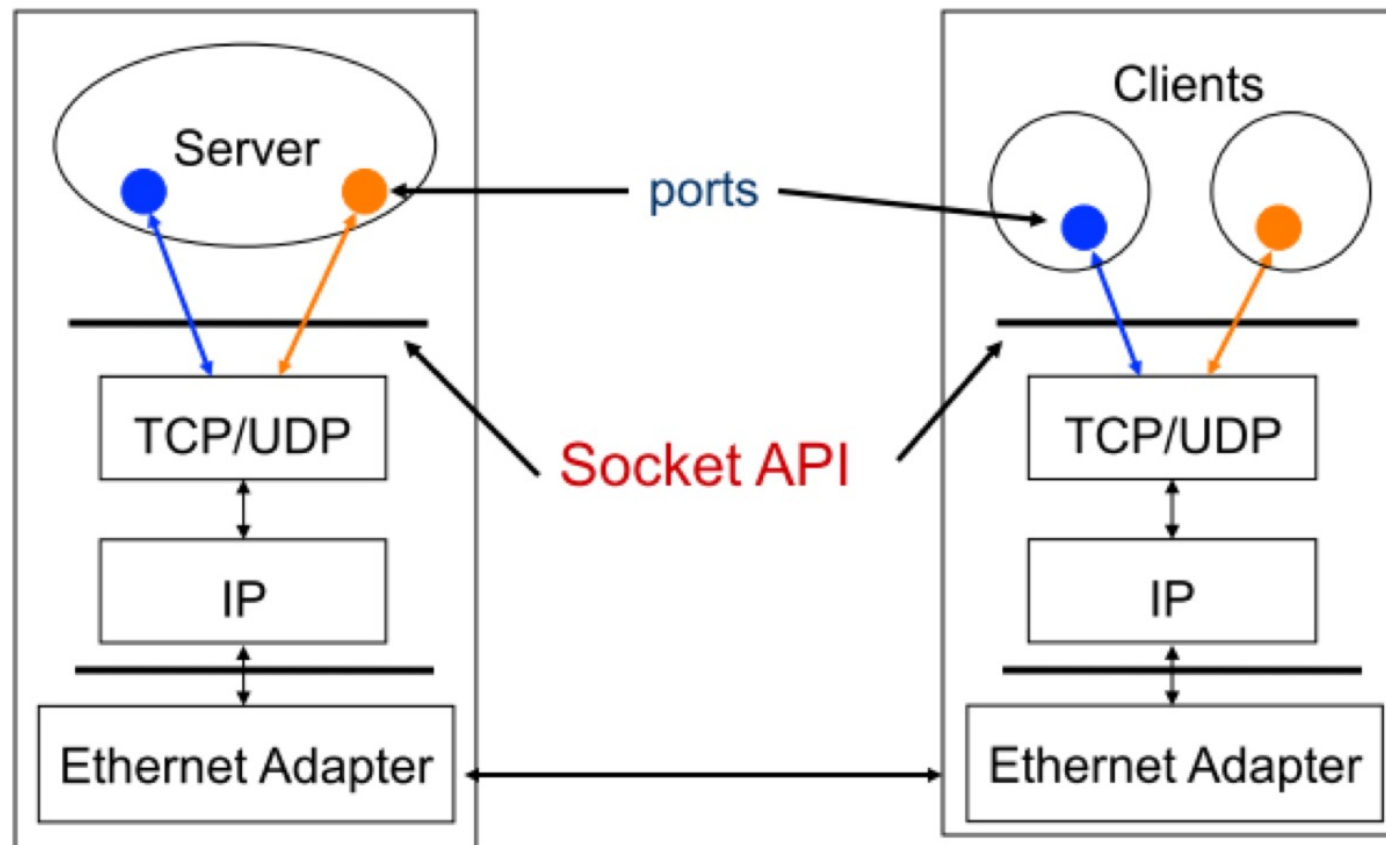


Alla trafik är paketförmädd, men det går att välja mellan:
TCP, som försöker efterlikna en fast förbindelse, kräver overhead,
UDP, som är best effort, med minimal säkerhet & overhead!

Sockets

LAB4

socket()
bind()
listen()

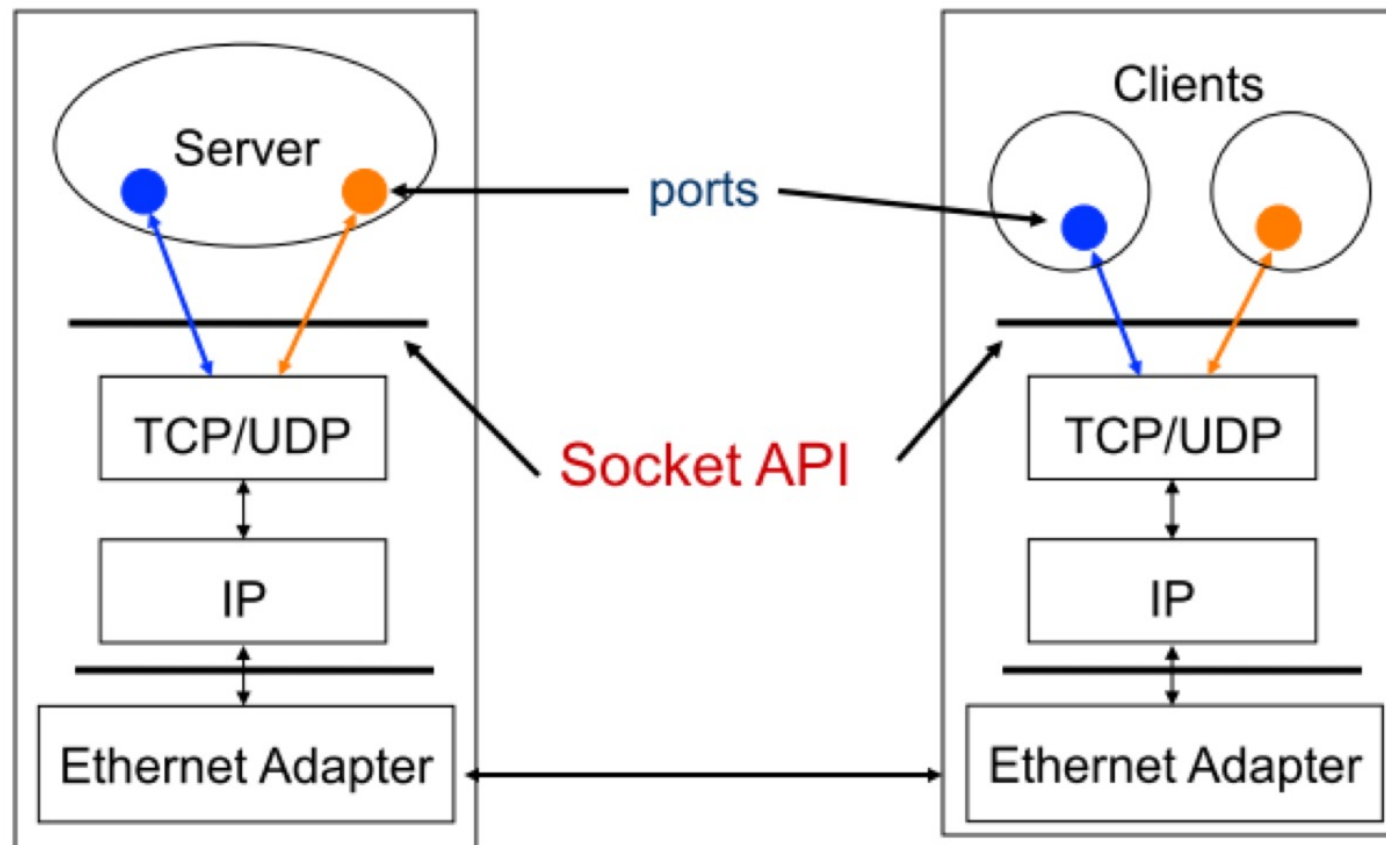


En tjänstetillhandahållande process behöver först en socket...
...som den "knyter" till en specifik dator/tjänst (bind)...
...därefter "lyssnar" (listen) den efter förfrågningar!

Sockets

LAB4

socket()
bind()
listen()



socket()
connect()

För att anropa en tjänst behövs en socket...
...därefter försöker klienten etablera
en förbindelse till servern (connect)!

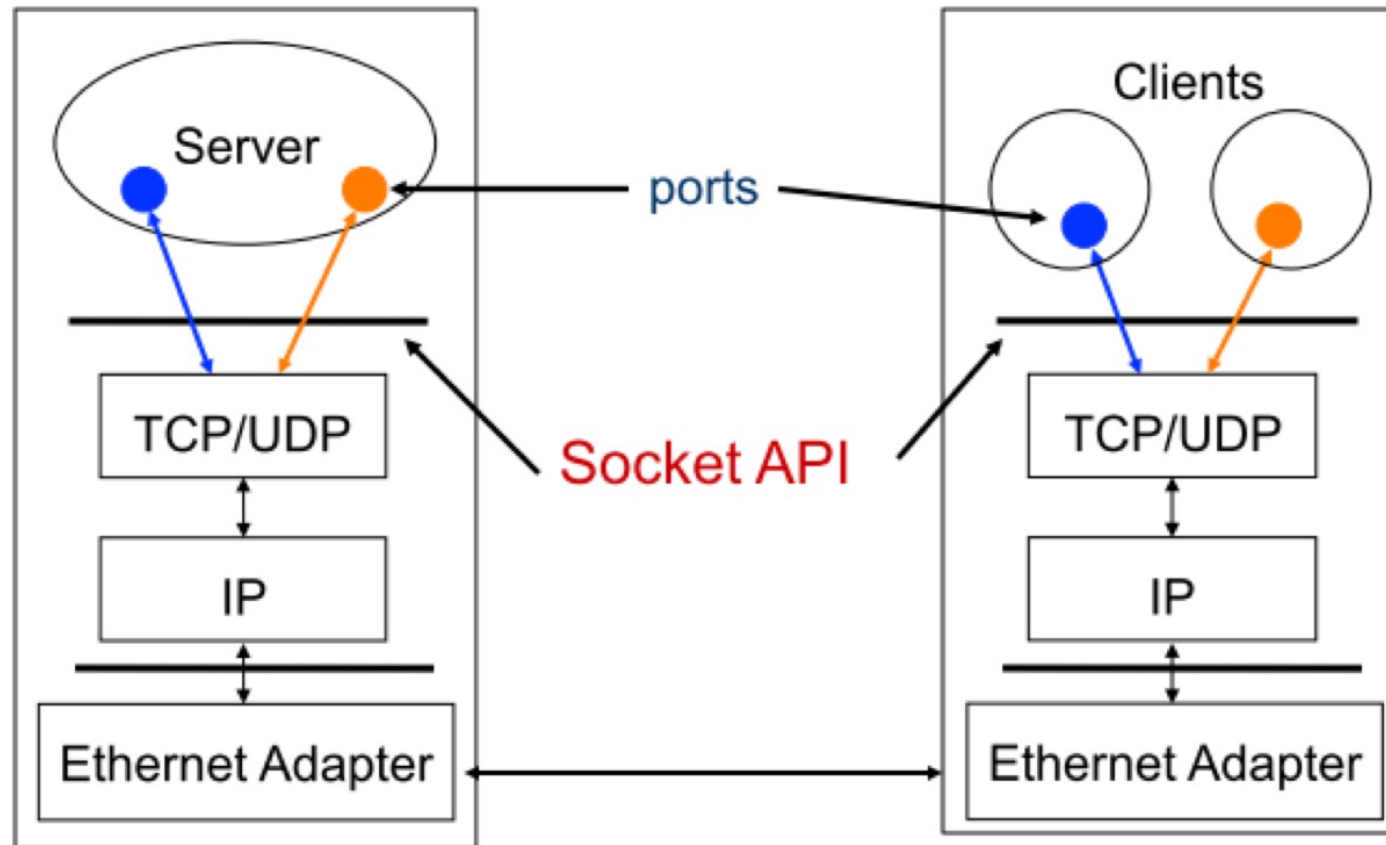
Sockets

LAB4

socket()
bind()
listen()

accept()
read()

write()
read()
close()



socket()
connect()

write()

read()
close()

Accepterar servern anropet kan data utbytas...
...tills klienten (eller servern) stänger förbindelsen...
...vilket servern märker och avslutar anropet.



LAB4

SERVER

```
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/in.h>
```

```
int main(int argc, char *argv[]) {
    int sock = socket(PF_INET, SOCK_STREAM, 0);
    if (sock == -1) {
        return 1;
    }
```

```
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    addr.sin_port = htons(9090);
    if (bind(sock, (const struct sockaddr *)&addr, sizeof(addr)) == -1) {
        return 2;
    }
```

```
    if (listen(sock, 5) == -1) {
        return 3;
    }
```

```
    for (;;) {
        int client = accept(sock, 0, 0);
        if (client == -1) {
            return 4;
        }
```

```
        char buf[1024];
        size_t len;
        while ((len = recv(client, buf, sizeof(buf), 0)) > 0) {
            write(STDOUT_FILENO, buf, len);
        }
```

```
    }
```

PF = Protocol Family
INET= IPv4
STREAM = TCP
DGRAM = UDP

AF = Address family
"Host to Network"

Max 5 anrop i kö!

client = ny fd -> ///

Normalt fork()/client



LAB4

CLIENT

```
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/in.h>
```

```
int main(int argc, char *argv[]) {
    int sock = socket(PF_INET, SOCK_STREAM, 0);
    if (sock == -1) {
        return 1;
    }

    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    addr.sin_port = htons(9090);
    if (connect(sock, (const struct sockaddr *)&addr, sizeof(addr)) == -1) {
        return 2;
    }

    char buf[1024];
    size_t len;
    while ((len = read(STDIN_FILENO, buf, sizeof(buf))) > 0) {
        if (send(sock, buf, len, 0) == -1) {
            break;
        }
    }
}
```

MÅSTE BYTAS UT!



Sockets

Vi nöjer oss med det i denna kurs, m.a.o...

...här tar vi fasta på att det går att etablera en mer eller mindre felfri förbindelse till valfri dator här på jorden...

...det centra är att det alltid kan gå fel: dator, disk, nätverk kan gå sönder så det måste kunna hanteras...

...samtidigt som det är styrkan, det ska kunna bli fel, och trots det ska systemet fungera tillfredställande - eller kanske bättre p.g.a. lastdelning (många maskiner hjälps åt)!

...förutom robusthet & prestanda (//) så är säkerhet också ett centralt begrepp...



Sockets

...du kommer att lära dig allt om länk/nätverkslagret i:

HE1033 Kommunikationsnät

HI2002 Routing i IP-Nät

...och mycket om transport/applikationslagret i:

HI1032 Kommunikationssystem

HI1031 Distribuerade Informationssystem

...förutom att kommunikation hanteras via ett os:api,
vad finns det för kopplingar till en operativsystemskurs?



Hur ska vi då använda möjligheten...

...då den uppenbart kan ge bl.a. prestanda & robusthet?

a) För att göra minnets fysiska plats transparent?

b) För att göra det icke-flyktiga minnets plats transparent?

c) För att göra en tjänst/funktions plats transparent?



Hur ska vi då använda möjligheten...

...då den uppenbart kan ge prestanda & robusthet?

a) För att göra minnets fysiska plats transparent?

Distributed Shared Memory (DSM)

Nix, svårt (omöjligt) att hantera (alla tänkbara) fel.

b) För att göra det icke-flyktiga minnets plats transparent?

c) För att göra en tjänst/funktions plats transparent?



Hur ska vi då använda möjligheten...

...då den uppenbart kan ge prestanda & robusthet?

a) För att göra minnets fysiska plats transparent?

Distributed Shared Memory (DSM)

Nix, svårt (omöjligt) att hantera (alla tänkbara) fel.

b) För att göra det icke-flyktiga minnets plats transparent?

Distributed File System (DFS)

Yes, tillgång från olika platser, enklare adm!

c) För att göra en tjänst/funktions plats transparent?



Hur ska vi då använda möjligheten...

...då den uppenbart kan ge prestanda & robusthet?

a) För att göra minnets fysiska plats transparent?

Distributed Shared Memory (DSM)

Nix, svårt (omöjligt) att hantera (alla tänkbara) fel.

b) För att göra det icke-flyktiga minnets plats transparent?

Distributed File System (DFS)

Yes, tillgång från olika platser, enklare adm!

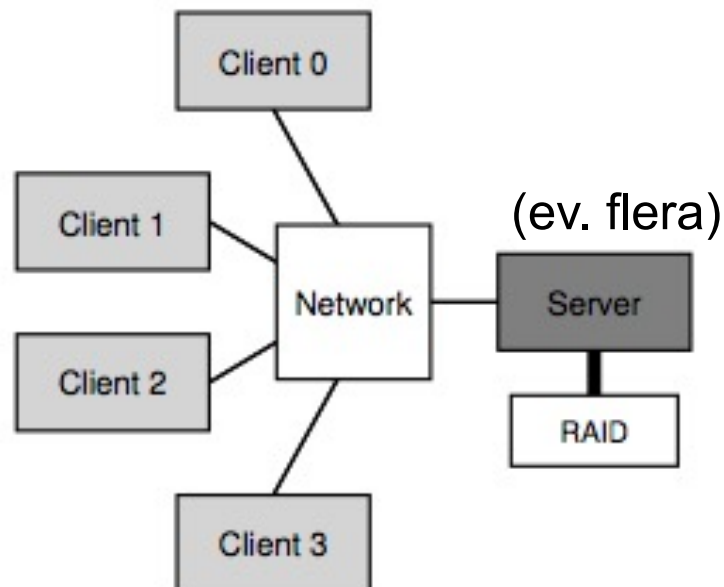
c) För att göra en tjänst/funktions plats transparent?

Remote Invocation (RI)

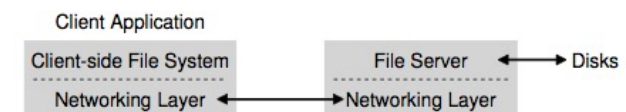
Yes, dra nytta av tjänster som finns någonstans!

b) + c) benämns ibland för Cloud computing
Låt oss ta en närmare titt på b) & c)...

b) Distributed File System (DFS)



- Platsoberoende access
- Filer kan delas
- Förenklad administration
- Bättre/sämre säkerhet
- (Sämre prestanda)
- Om möjligt – transparent!



En tidig lösning var Sun:s Network File System (NFS)...
...som lade grunden till en mycket viktig client/server princip...



Statelessness (historiken saknar betydelse)

...bakgrunden är rätt enkel: Då en krasch berör alla, så var en viktig designparametern snabb omstart, vilket bara kan ske om servern inte behöver komma ihåg vad som hänt!

Mao så måste varje anrop innehålla "alla" information som krävs för att utföra det som önskas. Servern kan t.ex. inte förväntas hålla information om mappning mellan filnamn och filpekare mellan anrop, då servern kan krascha och glömma mappningen alternativt klienten kan krascha och servern får aldrig reda på det och håller på en värdelös mappning!

Klienten, som svarar för kopplingen mot OS:ets file API, använder information dold i filpekaren för att verka stateful mot anropande programvara medan den är stateless mot filservern!

```

NFSPROC_GETATTR
  expects: file handle
  returns: attributes
NFSPROC_SETATTR
  expects: file handle, attributes
  returns: nothing
NFSPROC_LOOKUP
  expects: directory file handle, name of file/directory to look up
  returns: file handle
NFSPROC_READ
  expects: file handle, offset, count
  returns: data, attributes
NFSPROC_WRITE
  expects: file handle, offset, count, data
  returns: attributes
NFSPROC_CREATE
  expects: directory file handle, name of file, attributes
  returns: nothing
NFSPROC_REMOVE
  expects: directory file handle, name of file to be removed
  returns: nothing
NFSPROC_MKDIR
  expects: directory file handle, name of directory, attributes
  returns: file handle
NFSPROC_RMDIR
  expects: directory file handle, name of directory to be removed
  returns: nothing
NFSPROC_READDIR
  expects: directory handle, count of bytes to read, cookie
  returns: directory entries, cookie (to get more entries)

```

Client

```

fd = open("/foo", ...);
Send LOOKUP (rootdir FH, "foo")

```

```

Receive LOOKUP reply
allocate file desc in open file table
store foo's FH in table
store current file position (0)
return file descriptor to application

```

Server

```

Receive LOOKUP request
look for "foo" in root dir
return foo's FH + attributes

```

```

NFSPROC_GETATTR
  expects: file handle
  returns: attributes
NFSPROC_SETATTR
  expects: file handle, attributes
  returns: nothing
NFSPROC_LOOKUP
  expects: directory file handle, name of file/directory to look up
  returns: file handle
NFSPROC_READ
  expects: file handle, offset, count
  returns: data, attributes
NFSPROC_WRITE
  expects: file handle, offset, count, data
  returns: attributes
NFSPROC_CREATE
  expects: directory file handle, name of file, attributes
  returns: nothing
NFSPROC_REMOVE
  expects: directory file handle, name of file to be removed
  returns: nothing
NFSPROC_MKDIR
  expects: directory file handle, name of directory, attributes
  returns: file handle
NFSPROC_RMDIR
  expects: directory file handle, name of directory to be removed
  returns: nothing
NFSPROC_READDIR
  expects: directory handle, count of bytes
  returns: directory entries, cookie (to get next page)

```

read(fd, buffer, MAX);

Index into open file table with fd
 get NFS file handle (FH)
 use current file position as offset
 Send READ (FH, offset=0, count=MAX)

Receive READ request
 use FH to get volume/inode num
 read inode from disk (or cache)
 compute block location (using offset)
 read data from disk (or cache)
 return data to client

Receive READ reply
 update file position (+bytes read)
 set current file position = MAX
 return data/error code to app

```

NFSPROC_GETATTR
  expects: file handle
  returns: attributes
NFSPROC_SETATTR
  expects: file handle, attributes
  returns: nothing
NFSPROC_LOOKUP
  expects: directory file handle, name of file/directory to look up
  returns: file handle
NFSPROC_READ
  expects: file handle, offset, count
  returns: data, attributes
NFSPROC_WRITE
  expects: file handle, offset, count, data
  returns: attributes
NFSPROC_CREATE
  expects: directory file handle, name of file, attributes
  returns: nothing
NFSPROC_REMOVE
  expects: directory file handle, name of file to be removed
  returns: nothing
NFSPROC_MKDIR
  expects: directory file handle, name of directory, attributes
  returns: file handle
NFSPROC_RMDIR
  expects: directory file handle, name of directory to be removed
  returns: nothing
NFSPROC_READDIR
  expects: directory handle, count of bytes
  returns: directory entries, cookie (to get next)

```

```

read(fd, buffer, MAX);
  Index into open file table with fd
  get NFS file handle (FH)
  use current file position as offset
  Send READ (FH, offset=0, count=MAX)

```

```

Receive READ reply
  update file position (+bytes read)
  set current file position = MAX
  return data/error code to app

```



**Om...
förlorad/krasch
...sänd igen!**

```

Receive READ request
  use FH to get volume/inode num
  read inode from disk (or cache)
  compute block location (using offset)
  read data from disk (or cache)
  return data to client

```

Idempotent

```
NFSPROC_GETATTR
  expects: file handle
  returns: attributes
NFSPROC_SETATTR
  expects: file handle, attributes
  returns: nothing
NFSPROC_LOOKUP
  expects: directory file handle, name of file/directory to look up
  returns: file handle
NFSPROC_READ
  expects: file handle, offset, count
  returns: data, attributes
NFSPROC_WRITE
  expects: file handle, offset, count, data
  returns: attributes
NFSPROC_CREATE
  expects: directory file handle, name of file, attributes
  returns: nothing
NFSPROC_REMOVE
  expects: directory file handle, name of file to be removed
  returns: nothing
NFSPROC_MKDIR
  expects: directory file handle, name of directory, attributes
  returns: file handle
NFSPROC_RMDIR
  expects: directory file handle, name of directory to be removed
  returns: nothing
NFSPROC_READDIR
  expects: directory handle, count of bytes to read, cookie
  returns: directory entries, cookie (to get more entries)
```

close(fd);

Just need to clean up local structures
Free descriptor "fd" in open file table
(No need to talk to server)

Wow!



Bättre prestanda: Client-side Caching

Även ett snabbt nätverk är långsammare än datorns disk I/O, så vi tar till standardtricket och inför en client cache...

...första läsningen kostar nätverks I/O,
men därefter kan delar/hela filen finnas i client cache...

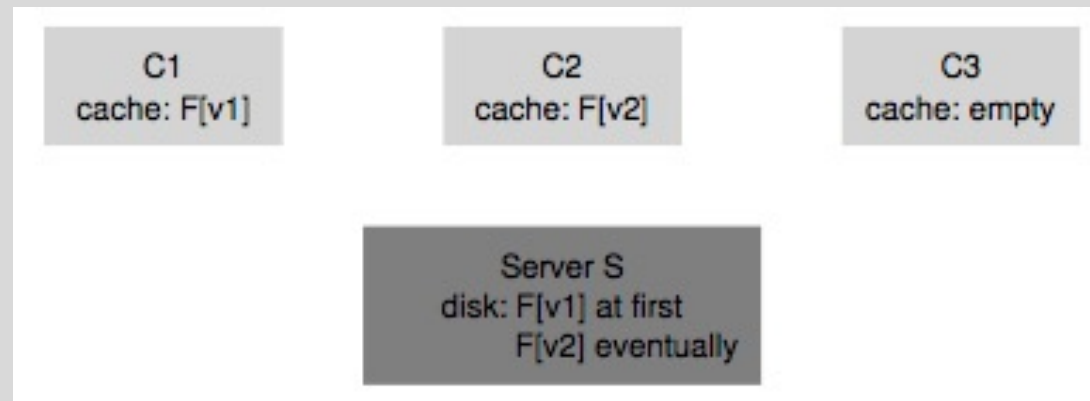
...en write skrivs först till cachen (snabbt!),
och efter en tid tillbaks över nätverket till file servern...

...dock kan det då uppstå ett problem med olika versioner,
något som måste hanteras och kallas Cache consistency...

...och består av problemen benämnda update visibility & stale cache!



Update visibility & stale cache...



Flush-on-close aka close-to-open, skriv tillbaks cache när filen stängs!
Före läsning, kontrollera om last write har ändrats (invalidate)!



The Andrew File System (AFS)

Det finns främst två problem med NFS...

- Det är svårt att veta om serverversionen är den senaste versionen?
- Otroligt många frågor om cachén är ok?



The Andrew File System (AFS)

Det finns främst två problem med NFS...

- Det är svårt att veta om serverversionen är den senaste versionen?
 - Hämta hela filen till det lokala filsystemet, skriv tillbaks hela filen (vid behov) till disken när den stängs! (Last close winn, det blir i alla fall inte ett lapptäcke.)
- Otroligt många frågor om cachen är ok?



The Andrew File System (AFS)

Det finns främst två problem med NFS...

- Det är svårt att veta om serverversionen är den senaste versionen?
 - Hämta hela filen till det lokala filsystemet, skriv tillbaks hela filen (vid behov) till disken när den stängs! (Last close winn, det blir i alla fall inte ett lapptäcke.)
- Otroligt många frågor om cachen är ok?
 - Begär notifieringar (call-back) om något ändrats i stället för att fråga hela tiden (jämför: att polla kontra avbrott)! Dock introducerar en server krasch mer trafik (ladda om cache) än i NFS där det i princip inte märktes.

Lösningar som succesivt har inkluderats i det populära/öppna NFS!



c) Remote Invokation

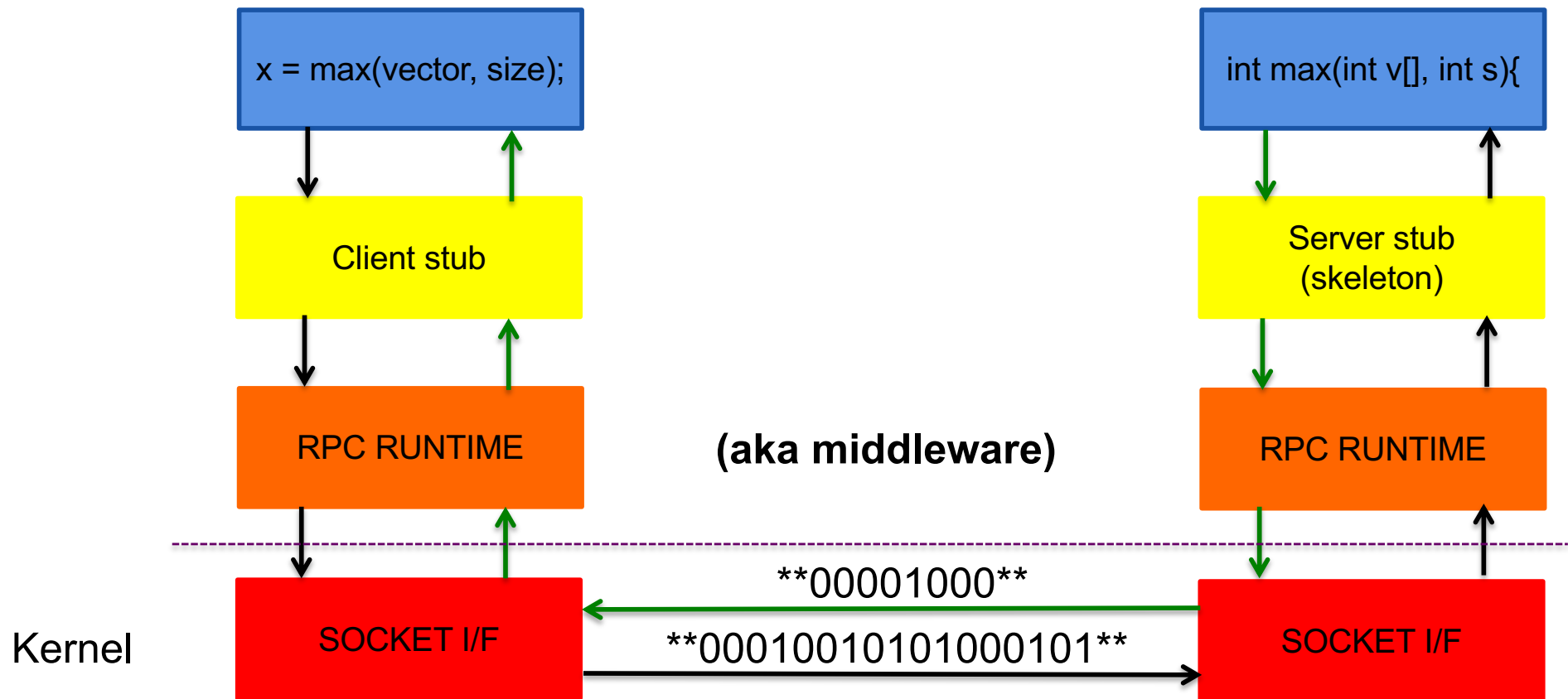
Att förädla de primitiva kommunikationsprimitiver som tillhandahålls i ett modernt os till att tillåta att funktioner/tjänster anropas på en annan dator är mer en programmeringsspråksabstraktion än en operativsystemsabstraktion...

...många varianter har varit i ropet under årens lopp, så i denna kurs ges endast en orientering kring två principiellt viktiga teknologier RPC & WebServices...

...området är så viktigt, och omfattande, så helheten får vänta till kursen **HI1031 Distribuerade Informationssystem!**

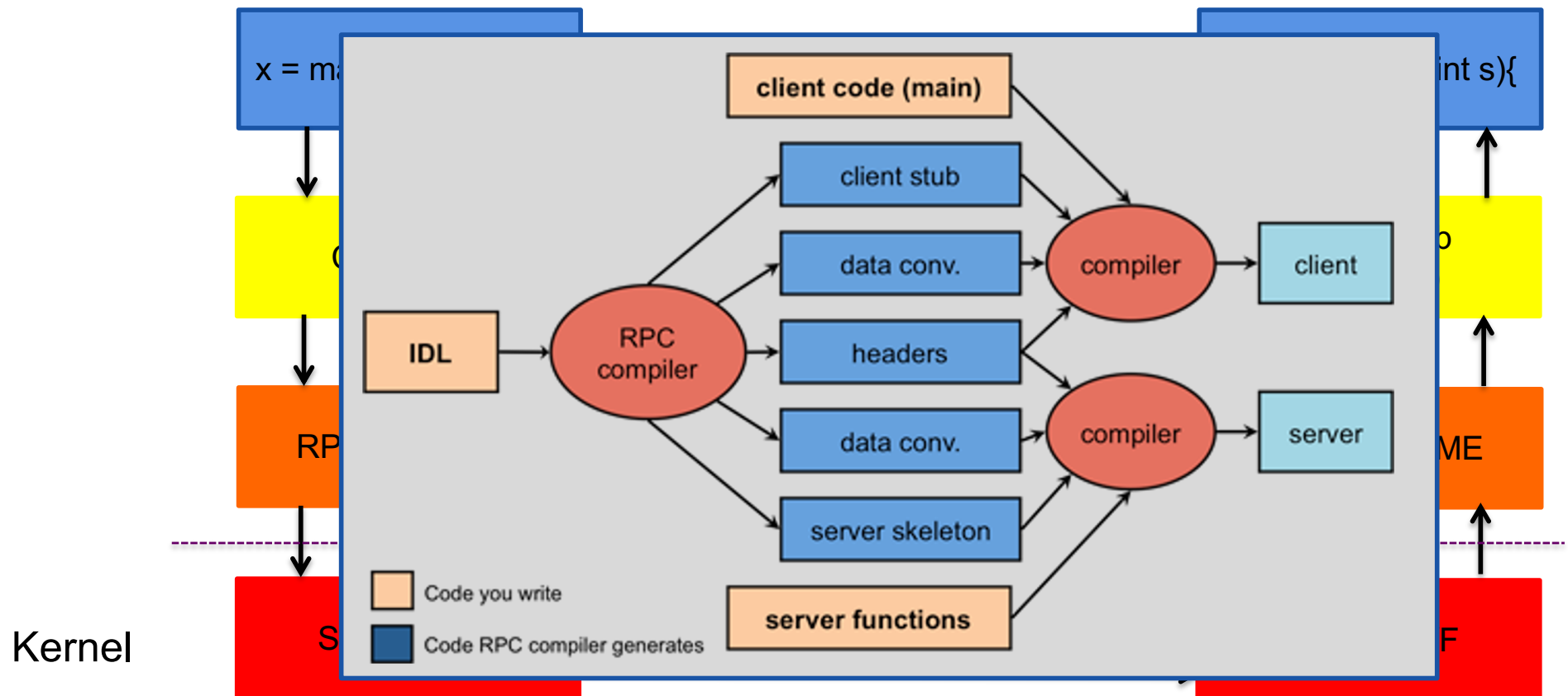


Remote Procedure Call (RPC)



Trafiken är "binär", det kan behövas en Registry/Look-up funktion.

Remote Procedure Call (RPC)



...bra, men så hände revolutionen med World Wide Web...



Steg #1:

DNS & World wide web (TCP/80)

LAB4



DNS = Domain Name System

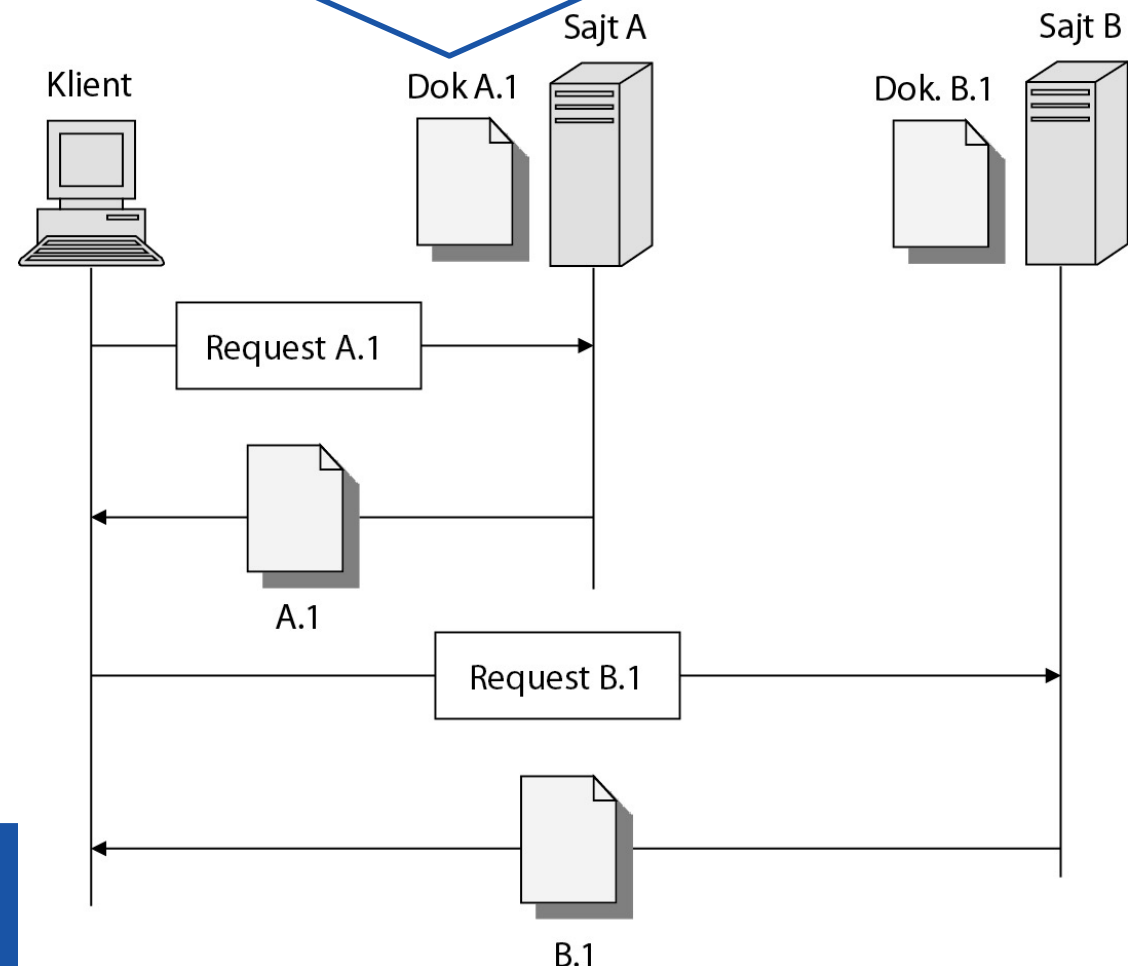
"Telefonkatalog"

HTTP = Hypertext Transfer Protocol

Hur en klient (Web browser) hämtar en **specifik fil** från en Web server!!!

HTML = Hypertext Markup Language

Hur innehållet i den filen ska vara **uppmärkt** för att klienten ska kunna presentera innehållet!



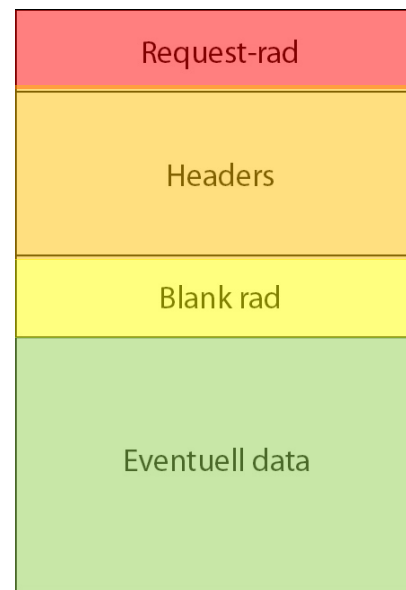
Statisk innehåll vs
Dynamiskt innehåll.



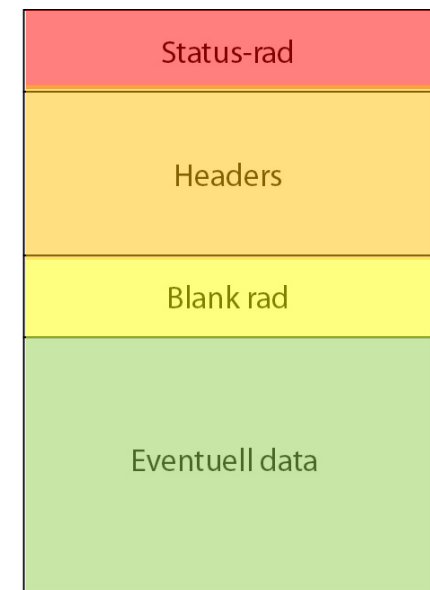
Steg #1: HTTP (TCP/80)

Status Code	Meaning	Description
200	Ok	The operation was successful.
201	Created	The create operation was successful.
204	No Content	The operation was successful and there is no content to return
304	Not Modified	The requested resource has not been modified.
400	Bad Request	The request is not in the required format.
401	Unauthorized	The user is not logged in.
402	Payment Required	A STAKE action failed due to insufficient funds.
403	Forbidden	Either the operation is not allowed on that resource or the user does not have sufficient rights.
404	Not Found	The requested resource does not exist.
405	Method Not Allowed	The request method is not allowed for this resource.
409	Conflict	The resource has been modified since the data was retrieved.
422	Unprocessable Entity	The server was unable to process the requested WIN or REFUND action.
500	Internal Server Error	An error occurred on the server while processing this request.
503	Service Unavailable	The service is not available for processing requests.

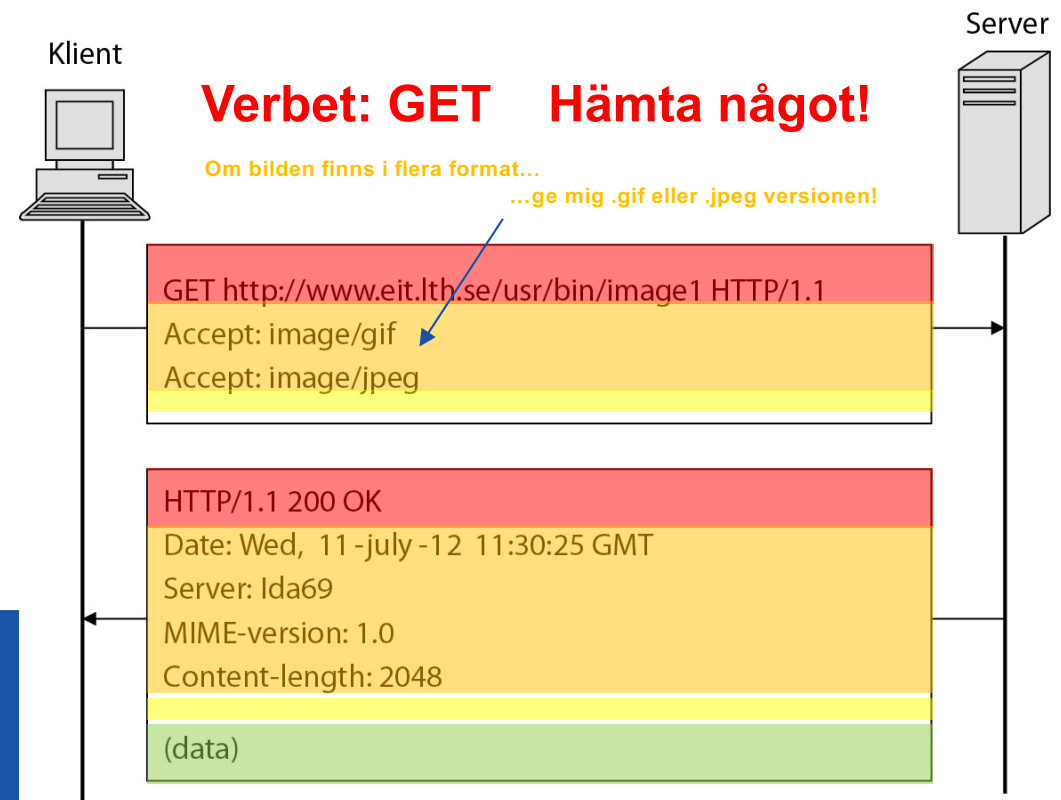
**"Modernt" protokoll,
allt är "läsbar" text!**



Request-meddelande



Response-meddelande

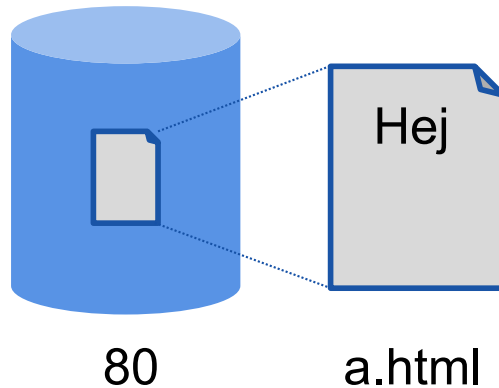




Steg #1: Telnet...

LAB4

217.210.53.181



```
vcxxs-macbook:~ vcxx$ telnet 217.210.53.181 80
Trying 217.210.53.181...
Connected to dsldevice.lan.
Escape character is '^['.
```

```
GET /a.html HTTP/1.1
Host: 217.210.53.181
```

Användaren skriver in detta för hand, och "låssas" vara en webbläsare!

```
HTTP/1.1 200 OK
```

```
Date: Mon, 26 Jan 2015 11:38:58 GMT
Server: Apache/2.2.26 (Unix) DAV/2 PHP/5.4.30
mod_ssl/2.2.26 OpenSSL/0.9.8za
Last-Modified: Mon, 26 Jan 2015 09:43:04 GMT
ETag: "3-50d8af6fd5a00"
Accept-Ranges: bytes
Content-Length: 3
MS-Author-Via: DAV
X-Frame-Options: SameOrigin
Vary: User-Agent
Content-Type: text/html
```

Den "inte ont anande" webbservern svarar och levererar eftersökt innehåll, filen a.html!

```
HejConnection closed by foreign host.
vcxxs-macbook:~ vcxx$
```

```
nc -v 130.237.5.214 8080
```



Steg #1:

HTTP kan mer...

Det är ok att addera information i url:en...

`http://server</path>/resource?usr="and"`

`http://server</path>/resource?usr="and"&psw="xyz"`

Body
Request/Response

No/Yes

GET

The GET method requests a representation of the specified resource.
Requests using GET should only retrieve data.

Opt/No

HEAD

The HEAD method asks for a response identical to that of a GET request, but without the response body.

Yes/Yes

POST

The POST method is used to submit an entity to the specified resource, often causing a change in state or side effects on the server.

Yes/Yes

PUT

The PUT method replaces all current representations of the target resource with the request payload.

Opt/Yes

DELETE

The DELETE method deletes the specified resource.

Opt/Yes

CONNECT

The CONNECT method establishes a tunnel to the server identified by the target resource.

Opt/Yes

OPTIONS

The OPTIONS method is used to describe the communication options for the target resource.

No/Yes

TRACE

The TRACE method performs a message loop-back test along the path to the target resource.

Yes/Yes

PATCH

The PATCH method is used to apply partial modifications to a resource.



Steg #2:

RESTful Web Services

LAB4

Hum...

...skulle det inte gå att använda HTTP för att enkelt nå en "annan typ av tjänsteserver" där främst GET/POST används för att göra "funktionsanrop liknande anrop i c"?

På servern `www.x.com`, finns "funktionen" `members`, som har parametern `id`. Anropet `www.x.com/members?id=2` ger svaret: `{ "2": "hobbe" }`

Tekniken benäms **RESTful Web Services**:

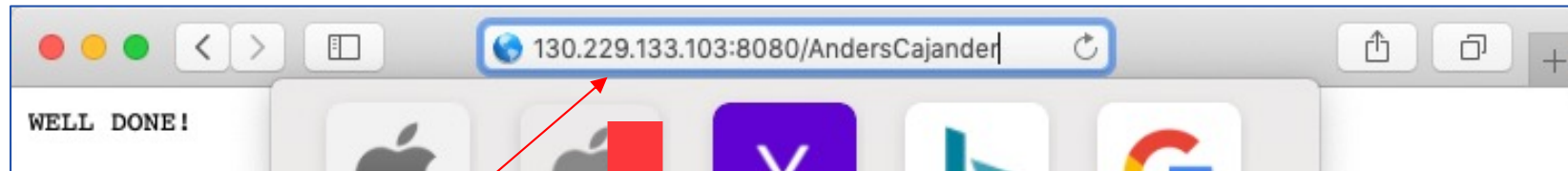
- Använder HTTP som protokoll
- Använder URI:er för att peka ut specifika tjänster
- Använder Javascript objekt (JSON) för att serialisera data.

REST stands for REpresentational State Transfer. It means when a RESTful API is called, the server will transfer to the client a **representation of the state of the requested resource**. There are several design criteria's, but for now will the key aspect be **statelessness**, the rest of the story in HI1031...



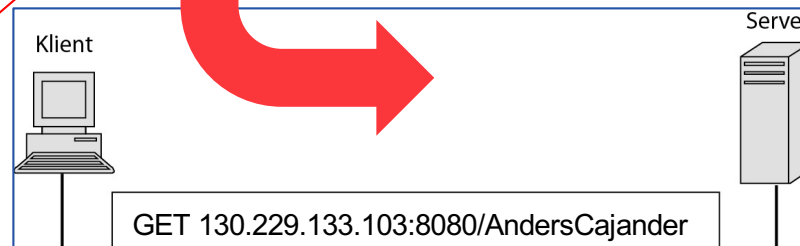
OBS! RÄTT IP-ADRESS:PORT
VIA CANVAS UNDER LABBEN!

Lab #4:A) Jag är med i matchen...



Du skriver det här...

...webbläsaren sänder:



```
*** (WORK PROC) GET /AndersCajander HTTP/1.1
Host: 127.0.0.1:8080
Upgrade-Insecure-Requests: 1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/14.1 Safari/605.1.15
Accept-Language: sv-se
Accept-Encoding: gzip, deflate
Connection: keep-alive
](368)
```

(Från en annan host!)

WELL DONE!

Webbservern svarar med: **HTTP/1.1 200 OK** ...som webbläsaren visa!

WELL DONE!



Lab #4:B) (Frivilligt) telnet/nc...

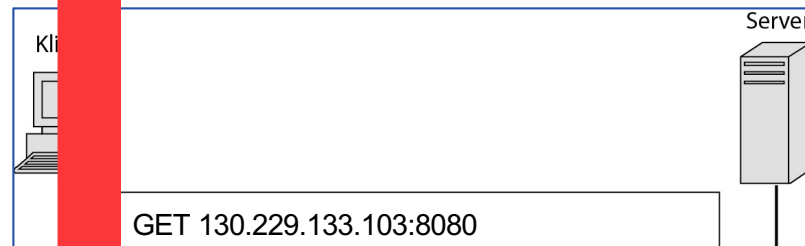
```
[n133-p103:~ ac$ nc 130.229.133.103 8080
```

```
HTTP/1.1 200 Ok!
```

```
WELL DONE!
```

```
n133-p103:~ ac$ █
```

Skriv exakt GET /...



```
*** (WORK PROC) [GET /AndersCajander HTTP/1.1
```

```
Host: 127.0.0.1:8080
```

```
Upgrade-Insecure-Requests: 1
```

```
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
```

```
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/14.1 Safari/605.1.15
```

```
Accept-Language: sv-se
```

```
Accept-Encoding: gzip, deflate
```

```
Connection: keep-alive
```

```
](368)
```

(Från en annan host!)

```
WELL DONE!
```



Lab #4:C) Egen klient!

```
Web Client 0.1
*** SOCKET Established...
*** CONNECT Success...
*** SENDING
[POST /api/1.0/pass/8533 HTTP/1.1
HOST: 127.0.0.1
Content-Type: application/json
Accept: application/json

{name : Anders}
]
*** SEND Success...
***[HTTP/1.1 200 Ok!

WELL DONE!
](32)
*** CONNECTION CLOSED!
```

OK

```
Web Client 0.1
*** SOCKET Established...
*** CONNECT Success...
*** SENDING
[POST /api/1.0/pass/8533 HTTP/1.1
HOST: 127.0.0.1
Content-Type: application/json
Accept: application/json

{
name : Anders}
]
*** SEND Success...
***[HTTP/1.1 400 Bad Request

Try again!
](40)
*** CONNECTION CLOSED!
*** DONE...
```

ERROR

```
*** (SERVER) CONNECTION ESTABLISHED [127.0.0.1]...
POST /api/1.0/pass/8533 HTTP/1.1
HOST: 127.0.0.1
Content-Type: application/json
Accept: application/json

{name : Anders}
*** (WORK PROC) CONNECTION CLOSED!
```



Årsklassen:
4 sista siffrorna
Ex: 8533

Gör om lab-kursen:
4 sista siffrorna + X
Ex: 8533X

Webbläsare ip:8080 ger lista med godkända!



Varför har vi inte ett distribuerat os?!

Linux, Mac OS & Windows är alla network operating systems...

...men det finns i dag inget os där det bara finns en instans av os:et som exekverar på otaliga nätverkade maskiner där en användare inte vet vilken processorkraft/minne/disk som faktiskt används från gång till gång...

...orsaken är nog tvådelad: De som utvecklar Linux, Mac OS & Windows vill nog inte sälja/underhålla "en" kopia av os:et som ska fungera på något generiskt HAL, samt att den enskilda användaren vill veta varför den egna datorn går långsamt!

Skynet begins to learn, at a geometric rate. It becomes self-aware at 2:14 a.m. Eastern daylight time, August 29.



Begrepp

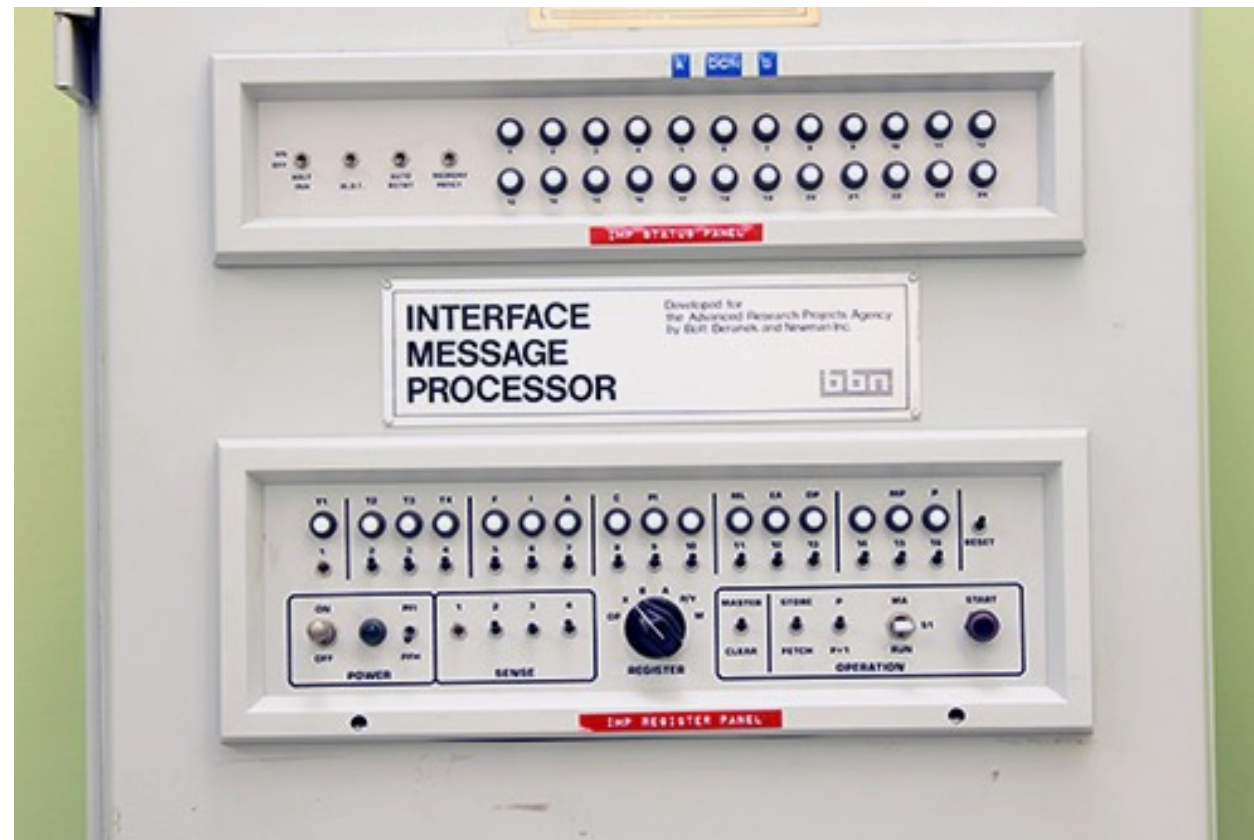
IPC

Shared memory
Mapped memory
Pipes
FIFO
Socket

TCP/UDP, Port number
IP, IP Address, URL, DNS
Socket(), Bind() & Listen()
Connect() & Accept()
Read(), Write() & Close()
Naming & Serialization

Distributed Shared Memory
Distributed File System
Remote Invocation

Statelessness & Idempotent
Update visibility & stale cache
RPC & Web Services



(En av Internets 4 första noder, en "IMP")



Historik

1811xx AC 0.8 Skapad, saknas övning.

210514 AC 0.9 REST-avsnittet uppdaterat.